

closure_evidence_20260923

BOB-233 — closure evidence (source layer): reload verify- after-write

Revision: 2 Last modified: 2026-09-23T17:25:00Z

Field	Value
Item	BOB-233 (Bug) — <code>start.sh --reload-jackett / --reload-proxy-go</code> printed success while the running container kept the OLD image
Status of item	still open. This document records source-layer + hermetic-stub evidence only.
Files changed	<code>start.sh</code> (<code>reload_go_service + new _go_service_image_matches</code>), <code>NEW tests/unit/test_start_reload_verifies_image.sh</code> , <code>tests/unit/test_start_reload_jackett.sh</code> (Rev 2, §11.4.120 reconciliation)
start.sh sha256 after fix	64706f27f97a6a1b8b47296807bb5f6f609142aafa7a3859dc1c10f17ea9d59f

Root cause (§11.4.102)

`reload_go_service()` treated `podman-compose up -d <svc> exiting 0` as proof that the container was recreated. `podman-compose` does not recreate a container whose compose config is unchanged, so after build retagged `localhost/boba_boba-jackett:latest` to the new image, the running container still pointed at the old image id. Nothing read the running image back, so the success line was printed unconditionally after `compose` exited 0.

Read-only probe of the live host (no restart, stop or recreate), taken while working on this fix. The stack had already been moved onto the new image by an earlier `--recreate`, so this probe shows how the ids are shaped, not the defect itself:

```
$ podman inspect boba-jackett --format '{{.Config.Image}}|{{.ImageName}}'|
localhost/boba_boba-jackett:latest|localhost/boba_boba-jackett:latest
$ podman image inspect localhost/boba_boba-jackett:latest --format '{{.Id}}'
38ef117ec563fc347e91573e289107b513bfb321c5d9177f8708516ab3547486
```

The container's `.Config.Image` gives the tag it was created from, and that tag now resolves to the fresh build. So container `.Image` compared with `image inspect <ref> .Id` is the right comparison for verify-after-write.

Fix

After `up -d <svc>, _go_service_image_matches` reads back:

- `$CONTAINER_RUNTIME container inspect --format '{{.Image}}' <svc>`, the image the container is running
- `$CONTAINER_RUNTIME container inspect --format '{{.Config.Image}}' <svc>`, then `image inspect --format '{{.Id}}' <ref>`, the image the fresh build produced

A leading `sha256:` is stripped from both ids (docker adds it, podman does not). The flow is then:

1. **Match:** print success, now including the verified image id.
2. **Mismatch:** print a warning showing both ids, then run a scoped `$REAL_COMPOSE_CMD up -d --force-recreate --no-deps <svc>` (the same compose tool `start.sh` already uses; no new raw runtime lifecycle command) and re-verify.
3. **Still mismatched:** `[ERROR]` showing both ids, `exit 1`, and no success line.
4. **Unreadable id** (missing container or empty inspect output): `[ERROR]` and `exit 1`. This follows §11.4.201(4): refuse conservatively, never assume success.

RED: the pre-fix artifact (HEAD start.sh)

```
== BOB-233 start.sh reload verify-after-write (--green) ==
  FAIL: STALE_UP_IS_FORCE_RECREATED – BOB-233 repro: stale `up -d`
  is detected and force-recreated onto the new image
    rc=0 running=2a3173ce1941 (want 38ef117ec563); force-
  recreate issued=0
  FAIL: STILL_STALE_FAILS_LOUD – still-stale after forced recreate
  -> non-zero exit, both image ids printed, no success line
    rc=0; output tail: [SUCCESS] boba-jackett image rebuilt
  [INFO] Recreating boba-jackett container from the new image
  (podman-compose up -d boba-jackett)... [SUCCESS] boba-jackett
  recreated – Go source changes are now live
  PASS: HEALTHY_UP_NO_FORCE – golden-FALSE: correct recreate ->
  success, no extra forced recreate
  PASS: SHA_PREFIX_NORMALIZED – golden-FALSE: sha256: prefix
  difference is not a mismatch
  FAIL: UNRESOLVABLE_FAILS_LOUD – unreadable running image id ->
  refuse, never claim success
    rc=0; output tail: [INFO] Recreating boba-jackett
  container from the new image (podman-compose up -d boba-
  jackett)... [SUCCESS] boba-jackett recreated – Go source changes
  are now live
  FAIL: PROXY_GO_VERIFIED – --reload-proxy-go gets the same
  verify+force, scoped to its own service
    rc=0 running=2a3173ce1941; log: boba-ctl.sh|config podman-
  compose|build|qbittorrent-proxy-go podman-compose|up|-d|
  qbittorrent-proxy-go
  RESULT: 2 passed, 4 failed
  exit=1
```

ANSI colour codes are stripped here for readability. The RED run reproduces the reported symptom: rc=0 and “now live” printed while the running container is still on 2a3173ce1941. The two golden-FALSE controls pass on the pre-fix artifact, as they should.

GREEN: the fixed artifact

```
== BOB-233 start.sh reload verify-after-write (--green) ==
  PASS: STALE_UP_IS_FORCE_RECREATED – BOB-233 repro: stale `up -d`
  is detected and force-recreated onto the new image
  PASS: STILL_STALE_FAILS_LOUD – still-stale after forced recreate
  -> non-zero exit, both image ids printed, no success line
  PASS: HEALTHY_UP_NO_FORCE – golden-FALSE: correct recreate ->
  success, no extra forced recreate
  PASS: SHA_PREFIX_NORMALIZED – golden-FALSE: sha256: prefix
  difference is not a mismatch
  PASS: UNRESOLVABLE_FAILS_LOUD – unreadable running image id ->
  refuse, never claim success
```

```
PASS: PROXY_GO_VERIFIED --reload-proxy-go gets the same
verify+force, scoped to its own service
RESULT: 6 passed, 0 failed
green exit=0
```

§1.1 paired mutations (--red), all killed

```
== BOB-233 start.sh reload verify-after-write (--red) ==
  RED-OK: STALE_UP_IS_FORCE_RECREATED -- killed by: s#up -d --
force-recreate --no-deps "$service"#up -d "$service"#
  RED-OK: STILL_STALE_FAILS_LOUD -- killed by: /after a forced
recreate/{n;n;s/exit 1/return 0/}
  RED-OK: HEALTHY_UP_NO_FORCE -- killed by: s#if !
_go_service_image_matches "$service"; then#if true; then#
  RED-OK: SHA_PREFIX_NORMALIZED -- killed by: s#_GO_BUILT_ID="\$
{_GO_BUILT_ID\#sha256:}"#:#
  RED-OK: UNRESOLVABLE_FAILS_LOUD -- killed by: /Cannot read back
the running image/{n;s/exit 1/return 0/}
  RED-OK: PROXY_GO_VERIFIED -- killed by: s/reload_go_service
"qbittorrent-proxy-go"/reload_go_service "boba-jackett"/
RESULT: 6 passed, 0 failed
red exit=0
```

Mutations are applied only to the sandbox copy of `start.sh`, never to the repository file.

Sibling suites after the fix (Rev 2)

The Rev 1 run found that `tests/unit/test_start_reload_jackett.sh` scored 9/10. `EXIT_ZERO_AND_LIVE_MSG` returned `rc=1` because the suite's stateless podman recorder returns empty output for `container inspect`, and the fixed code correctly refuses an image id it cannot read (§11.4.201(4)). In Rev 2 that suite was reconciled under §11.4.120. Only its podman shim changed: it is now a recorder that also answers the image read-back with consistent ids. So `EXIT_ZERO_AND_LIVE_MSG` now passes because the running and built ids really match, not because the check was relaxed. `podman-compose` is still the plain recorder, which keeps `BOBA_SHIM_FAIL` injection working. All 10 original checks and their mutations are unchanged. One new check was added: `MISMATCH_FAILS_LOUD`. Here the shim reports a running id different from the built id, and a recorder compose that never recreates leaves the container stale. The run must end non-zero, print no success line, and make exactly one `--force-recreate --no-deps` attempt.

The reload suites that stub podman for start.sh were found with grep -rln podman tests --include='*.sh' | xargs grep -ln start.sh, which listed test_start_reload_{jackett,verifies_image,recreate,python,plugins,harne ss}.sh and the ownership suites. Every test_start_reload_*.sh suite was run, with ANSI colour codes stripped:

```
### test_start_reload_jackett.sh
== start.sh --reload-jackett / --reload-proxy-go (--green) ==
    PASS: BUILD_ISSUED – rebuilds the boba-jackett image with the
    exact compose argv
    PASS: UP_ISSUED – recreates the boba-jackett container with the
    exact compose argv
    PASS: BUILD_BEFORE_UP – build is ordered BEFORE up (else up
    recreates from the stale image)
    PASS: UP_EXACTLY_ONCE – recreates exactly once
    PASS: NEVER_USES_BOBA_CTL_FOR_BUILD – bypasses boba-ctl for
    build (it has no build verb) – the root-cause fix
    PASS: NO_STACK_TEARDOWN – never issues a bare compose down/up
    (that is --recreate)
    PASS: EXIT_ZERO_AND_LIVE_MSG – exits 0 and reports the source is
    live
    PASS: BUILD_FAILURE_IS_FATAL – aborts without recreating when
    the image rebuild fails
    PASS: NO_RUNTIME_REFUSES – exits 1 with an honest error when no
    compose tool exists
    PASS: PROXY_GO_TARGETS_DIFFERENT_SERVICE – --reload-proxy-go
    targets qbittorrent-proxy-go, not boba-jackett
    PASS: MISMATCH_FAILS_LOUD – BOB-233: running image != fresh
    build and still stale after a forced recreate -> non-zero exit, no
    success line
RESULT: 11 passed, 0 failed
exit=0
### test_start_reload_jackett.sh --red
== start.sh --reload-jackett / --reload-proxy-go (--red) ==
    RED-OK: BUILD_ISSUED – killed by: s#\ $REAL_COMPOSE_CMD build "\
    $service"# $REAL_COMPOSE_CMD build "WRONGSERVICE"#
    RED-OK: UP_ISSUED – killed by: s#\ $REAL_COMPOSE_CMD up -d "\
    $service"# $REAL_COMPOSE_CMD up -d "WRONGSERVICE"#
    RED-OK: BUILD_BEFORE_UP – killed by: /print_info "Rebuilding \
    $service image/i \    $REAL_COMPOSE_CMD up -d "$service"
    RED-OK: UP_EXACTLY_ONCE – killed by: /are now live/a \
    $REAL_COMPOSE_CMD up -d "$service"
    RED-OK: NEVER_USES_BOBA_CTL_FOR_BUILD – killed by: s#\
    $REAL_COMPOSE_CMD build# $COMPOSE_CMD build#
    RED-OK: NO_STACK_TEARDOWN – killed by: /print_info "Rebuilding \
    $service image/i \    $REAL_COMPOSE_CMD up -d
    RED-OK: EXIT_ZERO_AND_LIVE_MSG – killed by: s/are now live/are
    stale/
    RED-OK: BUILD_FAILURE_IS_FATAL – killed by: /Failed to rebuild \
    $service image/{n;s/exit 1/:/}
```

```

RED-OK: NO_RUNTIME_REFUSES – killed by: s/if \[\[ -z "\$REAL_COMPOSE_CMD" \]\]; then/if false; then/
RED-OK: PROXY_GO_TARGETS_DIFFERENT_SERVICE – killed by: s/reload_go_service "qbittorrent-proxy-go"/reload_go_service "boba-jackett"/
RED-OK: MISMATCH_FAILS_LOUD – killed by: /after a forced
recreate/{n;n;s/exit 1/return 0/}
RESULT: 11 passed, 0 failed
exit=0
### test_start_reload_verifies_image.sh                RESULT: 6
passed, 0 failed exit=0
### test_start_reload_verifies_image.sh --red          RESULT: 6
passed, 0 failed exit=0 (6/6 RED-OK)
### test_start_reload_python.sh                        RESULT: 10
passed, 0 failed exit=0
### test_start_reload_recreate.sh                     RESULT: 11
passed, 0 failed exit=0
### test_start_reload_plugins.sh                      RESULT: 8
passed, 0 failed exit=0
### test_start_reload_harness.sh                      RESULT: 6
passed, 0 failed exit=0 (control-needle self-test)
bash -n start.sh: OK

```

The ownership suites

(test_ownership_{rootless_detection,precondition,repair}.sh) also match the grep. They stub podman for the ownership gate, not for reload_go_service, and they do not exercise the changed code path, so they were not re-run in this pass.

Honest gaps (§11.4.6)

- 1. RESOLVED in Rev 2.** The test_start_reload_jackett.sh check EXIT_ZERO_AND_LIVE_MSG, which Rev 1 left red, is now 11/11 after the §11.4.120 reconciliation described above. The full scripts/pre_build_verification.sh was NOT run in this pass, so invariant 30 being green is inferred from the per-suite runs, not observed.
- 2. No live-stack evidence.** Per instruction, the running containers were not rebuilt or recreated. This change is proven only against a stateful stub that models podman-compose's no-recreate behaviour. §11.4.108 layers 3 and 4 (a real --reload-jackett on the host, with the running image id read back against podman images) are still owed before BOB-233 can close.
- 3. Docker path not exercised.** The stub imitates podman argv, and the sha256: normalization is tested only through the stub. That docker compose up --force-recreate --no-deps and docker container inspect behave the same way is documented upstream behaviour, but it was not tested on this host (no docker here).

4. **Image-name drift.** The comparison resolves the tag in the container's own `.Config.Image`. If a future compose change renamed the built image, a stale container would still reference the old tag, which would still resolve to the old id. The check would then report a false match. Current naming is stable (`localhost/boba_<svc>:latest`); this edge case is untested.
5. **shellcheck not run.** `shellcheck` is not installed on this host. Only `bash -n start.sh` was run, and it passed.
6. **Not independently reviewed yet** (§11.4.142 / §11.4.209). The change is uncommitted, as instructed.

Live addendum (conductor, 2026-09-23 ~19:20 CEST)

Independent re-verification in the conductor shell:

```
test_start_reload_verifies_image.sh 6/0, test_start_reload_jackett.sh
11/0, bash -n start.sh OK; RED on the pre-fix start.sh (git stash of
start.sh only) = RESULT: 2 passed, 4 failed; fix restored, GREEN again
6/0. Real run on the live stack: ./start.sh --reload-jackett ->
[SUCCESS] boba-jackett recreated on image 38ef117ec563 (verified
running) – Go source changes are now live; podman inspect boba-
jackett image 38ef117ec563fc347e91573e289107b513bfb321 == freshly
built boba_boba-jackett image 38ef117ec563; all four containers
healthy afterwards. Honest boundary: the live run exercised the
MATCH branch (ids were already equal from the prior recreate); the
MISMATCH -> scoped force-recreate -> loud-fail branches are proven
by the stateful-stub tests and mutations only, not on the live host.
Docker path untested (no docker here); no shellcheck available.
```